A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

Abstract

Explanation requires ground truth: to verify an account of a system, we must know its inner functioning—precisely what is missing wherever explainable AI (XAI) is most needed. Today's study objects are either simple, with a known but trivial mechanism, or genuinely complex, with no ground-truth mechanism against which to check explanations. We remove this dichotomy with a real computer architecture that is fully specified and differentiable. We re-implement the Atari 2600 Video Computer System (VCS) as two independent emulators, JUTARI (Julia) and JAXTARI (JAX), validated bit-for-bit against XITARI: 64/64 byte-identical RAM and 64/64 pixel-identical screens on tested Arcade Learning Environment (ALE) trajectories. A full-system soft step executes the exact hard machine and supplies surrogate gradients through CPU registers, RAM, and ROM bytes. We prove forward equivalence by construction and measure zero soft-handler disagreements in JUTARI (64 games) and JAXTARI (9 games spanning the exercised mapper families). Exact interventions on the hard machine independently test these gradients. A companion study finds that vanilla saliency can fail this test, while interventions recover more of the causal structure. GPU batching reaches $\sim$3.1M steps/s for the soft CPU core, not the full-system step. Both ports will be released under the MIT license upon acceptance.

1 Introduction

When we explain a system—in explainable AI (XAI) as anywhere else—we claim that *this* part is responsible for *that* behaviour. Checking such a claim requires knowing the inner functioning independently, as ground truth. Without it we can ask whether an explanation is plausible or convincing, but not whether it is *true*. Ground truth is the prerequisite for verification, and it is exactly what is missing where explanation matters most.

The systems available to study fall into two unsatisfying camps. Simple, procedural models—decision trees, rule lists, sparse linear models—have fully known inner functioning, but there the explanation is essentially the model itself, testing an XAI method on nothing it could get wrong. The systems XAI does target—deep neural networks, learned agents, real-world pipelines—are genuinely complex and have no independently known mechanism, so an attribution can be plausible and wrong with no way to tell (Atrey, Clary, and Jensen 2020; Nikulin et al. 2019).

Neuroscience faces a similar problem. Jonas and Kording (2017) asked whether the standard systems-neuroscience toolkit—connectomics, tuning curves, lesioning, dimensionality reduction—could recover how a MOS 6502 microprocessor computes. The chip was chosen because its mechanism is complex and completely known. Even with perfect data the methods produced meaningful-looking structure but did not recover the actual organisation. This suggests testing methods against known mechanisms. Yet the processor was not differentiable, excluding modern gradient-based XAI. The MOS 6502 is the CPU at the heart of the Atari 2600 VCS, the platform on which modern deep reinforcement learning was demonstrated (Mnih et al. 2013, 2015; Bellemare et al. 2013).

We therefore build a study object that is *complex*, *fully specified*, and *differentiable*. The VCS combines a 6507 CPU executing cartridge ROM, a Television Interface Adapter (TIA) generating pixels cycle by cycle, and a RIOT chip providing RAM, timers, and input. We implement two independent ports: JUTARI in Julia (Bezanson et al. 2017) and JAXTARI in JAX (Bradbury et al. 2018). Both have bit-exact *hard* execution and differentiable *soft* CPU handlers, checked against XITARI (DeepMind 2016), the Stella-derived C++ reference (Stella Team 2024) used by the original DQN work.

Our contributions are:

- **Two VCS ports.** Independent Julia and JAX implementations reproducing reference RAM and screens on all 64 tested ALE games.
- **Exact-forward differentiation.** A full-system STE step (Theorem 1) and bounds for relaxed execution (Theorem 2).
- **Mechanism-grounded evaluation.** Exact interventions test surrogate attributions to ROM, RAM, registers, and inputs.
- **AI-assisted implementation.** Approximately 137 hours of active development time, with agent-written code checked by three conformance harnesses.
- **Reproducible measurements.** Bit- and pixel-exactness, gradient checks, and CPU/GPU throughput with explicit scope.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

2 Background and Related Work

The dominant XAI tools are *attribution* or *saliency* methods, which assign each input element a number reflecting how much it mattered to the output. Grad-CAM computes gradients with respect to convolutional features (Selvaraju et al. 2020); Grad-CAM++ refines their weighting (Chattopadhyay et al. 2018). Their limitation is verification: conventional faithfulness metrics "need not correlate with the actual factors responsible for the network's decision" (Chattopadhyay et al. 2018). Validation often relies on proxies such as bounding-box overlap or human trust.

The problem sharpens for reinforcement learning (RL) agents. DQN learns Atari games from pixels (Mnih et al. 2013, 2015). Greydanus et al. (2018) introduced perturbation saliency and described strategies such as Breakout "tunneling", but verified one case using a deliberately inserted feature. Nikulin et al. (2019) compared built-in saliency with human eye-tracking, a behavioural surrogate. Through counterfactual interventions, Atrey, Clary, and Jensen (2020) showed that Atari saliency explanations frequently fail and should be treated as exploratory. Such et al. (2019) enabled comparisons of trained agents without resolving mechanism-grounded validation.

Surveys repeatedly identify this gap (Qing et al. 2022; Vouros 2023; Cheng, Yu, and Xing 2025; Saulières 2025). XDQN trains a transparent surrogate but measures fidelity to another black box (Kontogiannis and Vouros 2023). Other work supplies known structure through inserted features (Yang and Kim 2019), or small transformers compiled or trained for interpretability (Lindner et al. 2023; Gupta et al. 2024). Tracr and InterpBench use known circuits as targets. Our target is instead a real human-written program, never designed for interpretability, executing across CPU, memory, control flow, bank switching, timers, and cycle-timed graphics. We evaluate explanations against exact interventions on machine variables, rather than recovery of a designed circuit. This extends the test proposed by Jonas and Kording (2017) to gradient-based XAI.

Known Operators and Differentiable Programming

Our approach relates to *known-operator learning*, introduced in deep-learning computed tomography (Würfl et al. 2016). Maier et al. (2019b) showed that embedding known differentiable operators removes their error contributions from a network's bound. The same principle appears in physics-informed neural networks (Raissi, Perdikaris, and Karniadakis 2019) and operator-aware deep learning (Maier et al. 2019a). In the VCS, every operator is known; the task is to explain how the fixed program maps inputs to outputs.

Julia's Zygote provides source-to-source automatic differentiation (Innes 2019); JAX provides trace-based differentiation and XLA compilation (Bradbury et al. 2018). We represent ROM as a weight tensor, RAM as an addressable tape (Graves, Wayne, and Danihelka 2014), and discrete decisions using deterministic softmax and straight-through estimators (Bengio, Léonard, and Courville 2013). Only the $T \to 0$ relation is shared with Gumbel-softmax (Jang, Gu, and Poole 2017); no Gumbel noise is used.

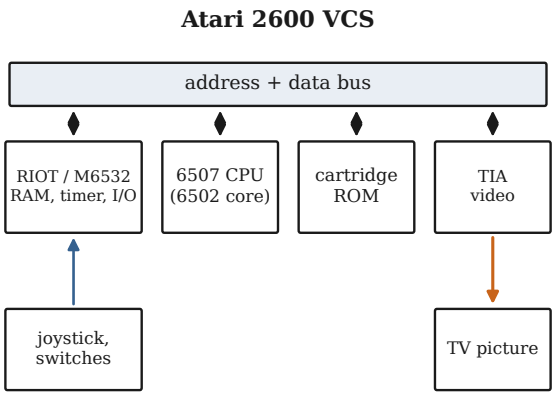


Figure 1: The Atari 2600 VCS. A 6507 CPU executes cartridge ROM over a shared bus. The RIOT provides RAM, timer, and I/O; the TIA generates cycle-timed graphics. Audio is not implemented in our ports.

Why the Atari VCS Is Good Ground Truth

The VCS (Figure 1) is a real computer architecture, small enough to know completely yet complex enough to be interesting. Its 6507 CPU (a cost-reduced MOS 6502) has a 13-bit address space; larger cartridges *bank-switch* ROM pages. The RIOT (M6532) provides 128 bytes of RAM, an interval timer, and joystick, paddle, and console-switch I/O. The TIA has no frame buffer: the CPU writes registers while the electron beam scans the screen. These include sprite colours `COLUP0`, `COLUP1`, playfield colour `COLUPF`, background `COLUBK`, and sprite-shape and position registers. The TIA produces screen and audio signals; our ports implement the graphics path only.

ALE (Bellemare et al. 2013) exposes the VCS to RL agents. XITARI (DeepMind 2016), derived from Stella (Stella Team 2024), is the DQN reference. Given a ROM, action stream, and deterministic initial state, its registers, RAM, and pixels provide an independent reference for the ports.

Other Atari substrates and differentiable simulators. CuLE provides GPU Atari execution but is neither bit-exact nor differentiable (Dalton, Frosio, and Garland 2020). OCAtari and JAXAtari offer structured, GPU-oriented environments (Delfosse et al. 2023, 2026), but JAXAtari reimplements game logic rather than executing original ROMs. Brax differentiates continuous rigid-body physics (Freeman et al. 2021); we differentiate selected operations of a discrete computer while preserving its hard execution. We use ALE's game catalogue and evaluation protocol (Bellemare et al. 2013; Machado et al. 2018).

3 Building Two Differentiable VCS Ports

We ported XITARI subsystem by subsystem—CPU, bus, TIA, RIOT, cartridge mappers, console, controllers, and ALE wrapper—under three conformance harnesses. PXC1 compares per-instruction CPU, RAM, and TIA traces with XITARI; PXC2 cross-checks the independent ports; PXC-S compares full 210×160 frame buffers. Each port supports 151

Table 1: Port statistics and instruction throughput. CPU: Apple M1 Max; GPU: GTX 1080 Ti / Quadro RTX 5000, peak batch $N \approx 4096$. Batched JAX rows use the soft CPU core with a simplified bus.

	JUTARI	JAXTARI
Language / AD	Julia / Zygote	JAX / XLA
Source lines	8,630	10,111
Test lines	6,052	11,855
Test cases	∼1,183	803
Opcode bytes	189	189
Cartridge mappers	10	10
RAM-exact (of 64)	**64**	**64**
Pixel-exact (of 64)	**64**	**64**
Soft CPU core, 1 env (CPU)	**370,100**	1,178
Soft CPU core, batched CPU	—	∼60,000
Soft CPU core, batched GPU	—	**∼3.1 M**
Full-system soft step (CPU)	∼146,000	—

Table 2: Execution modes and gradient methods.

Mode	Forward	Backward	Used for
HARD	Exact	None	Reference
SOFT-STE	Exact, full	Surrogate	CPU/RAM/ROM
FULL	Exact in corner	Relaxed	α/T study
LEAKY	Mild divergence	Relaxed	Boundary test
NAIVE	Exact	Zero on indices	Baseline
SAMPLER	Exact	Bilinear	Position

documented NMOS 6502 instructions, the undocumented `USBC` alias, and 37 common illegal opcodes (189 opcode bytes), plus ten cartridge mappers: 2K, 4K, F8, F6, F4, three Superchip variants, E0, and FE. Seven mappers are exercised by the 64 games. Table 1 summarises the ports.

Hard and Soft Execution

In HARD mode the CPU uses integer arithmetic, bit flags, and opcode dispatch. The full-system SOFT-STE step, implemented in both ports, first executes the instruction on the hard CPU, bus, TIA, RIOT, and cartridge. It then replays the instruction through differentiable handlers against that same hard machine. ROM reads follow the hard cartridge's current bank/slice mapping; TIA, RIOT, and Superchip register reads are constants from the hard bus. Writes outside RAM affect only hard devices. Every CPU register and RAM cell is joined with its hard value through STE. Device and framebuffer states remain hard and receive no gradient. Gradients reach registers, RAM, and ROM bytes.

Table 2 distinguishes execution from gradient methods. FULL relaxes branches and reads without STE, rounding their results to integers; opcode dispatch remains hard. LEAKY uses $\alpha = 5.5$, $T = 0.145$ just outside the sufficient exactness region and permits mild forward divergence. NAIVE differentiates through hard discrete indices, giving zero position gradients. SAMPLER adds a bilinear position kernel.

The soft formulation uses three constructions. First, ROM and RAM reads use a one-hot address vector. For memory $r \in \mathbb{R}^M$ and address a,

$$\mathrm{peek}(r, a) \;=\; \mathbf{1}_a^\top r \;=\; \sum_i [\![i = a]\!]\, r_i \;=\; r_a. \qquad (1)$$

The value is r_a, with gradient $\partial\,\mathrm{peek}/\partial r = \mathbf{1}_a$. This is the discrete limit of attention-style memory addressing (Graves, Wayne, and Danihelka 2014).

Second, a general soft selection over $K = 256$ opcode candidates has logits ℓ and weights $w = \mathrm{softmax}(\ell/T)$:

$$\mathrm{select}(\ell, V; T) = w^\top V, \qquad (2)$$

where V contains candidate outputs. The selection approaches the hard winner as $T \to 0$ (Jang, Gu, and Poole 2017). This is deterministic, without Gumbel noise. Actual opcode dispatch remains a hard switch in both SOFT-STE and FULL. Figure 3 visualises the spread of soft selection using sprite columns, not opcodes.

Third, conditional branches use a sigmoid gate. Let $f \in [0, 1]$ be a relaxed status flag, $z = s(2f - 1)$ with $s = \pm 1$ for branch-when-set/clear, and δ the relative offset:

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1 - g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + g\,\delta, \qquad (3)$$

where $\mathrm{pc}_{\bar{b}}$ is the fall-through counter. FULL rounds the target to an integer. With an exact binary flag, a branch rounds correctly when $|\delta|\sigma(-\alpha) < 1/2$; thus $\alpha \geq 6$ suffices for 6502 offsets $|\delta| \leq 128$. FULL memory reads instead use distance-softmax at temperature T and round their byte values. Proposition 1 gives both exactness conditions.

The straight-through estimator (STE) (Bengio, Léonard, and Courville 2013) joins soft and hard values:

$$\mathrm{STE}(\mathrm{soft}, \mathrm{hard}) = \mathrm{soft} + \mathrm{sg}(\mathrm{hard} - \mathrm{soft}), \qquad (4)$$

where sg stops gradients. Its forward value is hard; its derivative follows soft. Applied after every instruction to all CPU registers and RAM cells, it makes the complete forward state exact (Theorem 1), independently of the relaxation parameters.

The key distinction is between *content* and *indices*. A stored hard switch routes gradients to selected values, as in max-pooling: register contents, sprite colours, graphics bits, and priority compositing. It does not differentiate which index was selected, such as a sprite column determined by strobe timing. A triangular (bilinear) sampler (Jaderberg et al. 2015) supplies a position surrogate. Whether that gradient reflects the actual mechanism must be tested by exact interventions.

4 Soft Equals Hard: Equivalence and Gradients

Let Φ_{H} and Φ_{S} be the hard and full-system SOFT-STE one-step maps on the complete machine state x (CPU, RAM, TIA, RIOT, cartridge, frame buffer). Proofs are in the supplement.

Theorem 1 (Exact forward equivalence)**.** *For every reachable state x and finite $\alpha, T > 0$, $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$. Consequently, identical initial states and inputs yield identical complete trajectories for any number of steps. SOFT-STE differs only in its surrogate backward rule.*

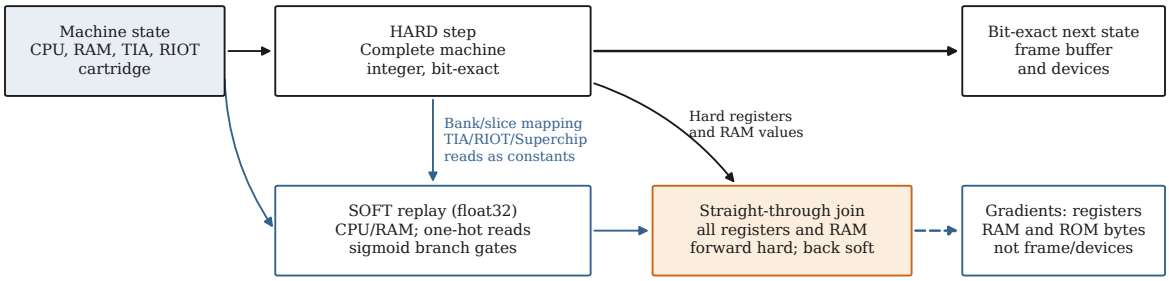


Figure 2: HARD executes each instruction on the complete machine bit-exactly. SOFT replays the same instruction using differentiable CPU/RAM handlers, soft state, and hard device-read constants. A straight-through join preserves hard register and RAM values while propagating surrogate gradients to registers, RAM, and ROM bytes. Frame-buffer and device states carry no gradient.

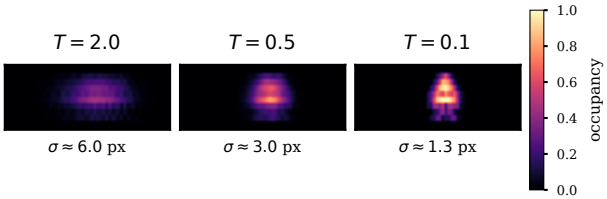


Figure 3: Temperature controls soft selection in pixel space. A sprite column is sampled 100 times and the hard renders averaged. Increasing T spreads occupancy ($\sigma \approx 1.3, 3, 6$ px). These averages illustrate temperature spread, not the bit-exact STE forward pass.

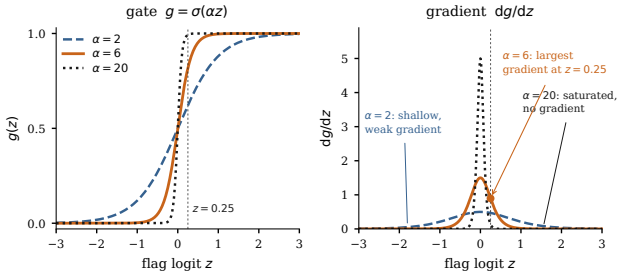


Figure 4: Sigmoid branch gate and derivative for $\alpha \in \{2, 6, 20\}$, with $z = 0.25$ marked. Small α gives a shallow gate; large α saturates it. Here $\alpha = 6$ gives the largest local gradient at the marked point.

Proof sketch. The full-system step executes the complete hard machine and replays the instruction through soft CPU/RAM handlers against that same state. Every CPU register and RAM cell is joined with its hard value by (4), while all device and frame-buffer states remain hard. Thus every output component equals the hard value, and trajectory equality follows by induction. □

Exact forward execution does not imply exact gradients: the STE derivative remains a surrogate of the discrete machine.

The FULL variant instead relaxes only branches and memory reads without STE, rounding their results to integers; opcode dispatch remains hard.

Assumption 1 (Decision margins)**.** *Every relaxed branch on the executed trajectory has flag-logit margin $m_t = |z_t| > 0$, with $m_{\min} = \min_t m_t > 0$. Each memory read selects a unique integer address, with other candidates at distance at least one.*

For exact processor flags, $z_t = \pm 1$ and $m_{\min} = 1$.

Theorem 2 (Temperature-limit bound)**.** *For the unrounded relaxed branch and read values, under Assumption 1, the local deviation from the hard operation satisfies*

$$\left\|\Phi_{\mathrm{S}}^{T,\alpha}(x) - \Phi_{\mathrm{H}}(x)\right\|_{\infty} \le C\left[e^{-\alpha m_{\min}} + \rho e^{-c/T}\right], \quad (5)$$

where C bounds value ranges, ρ the number of reads per step, and $c > 0$ the read-distance gap. The two rates remain separate: $\mathcal{O}(e^{-\alpha m_{\min}} + e^{-c/T})$. There is no opcode-dispatch term because dispatch remains hard.

Proof sketch. The sigmoid branch error is bounded by $e^{-\alpha m_{\min}}$; incorrect-address weight in a distance-softmax read has a geometric tail of order $e^{-c/T}$. Bounding the corresponding value differences gives (5). □

Rounding gives a stronger finite-parameter result.

Proposition 1 (Finite-parameter exactness)**.** *If every relaxed branch target and memory-read value is within $1/2$ of its hard integer value, rounding restores that value exactly. With hard opcode dispatch, the complete trajectory is then bit-exact.*

For a branch with binary flag and offset δ, the condition is

$$|\delta|\sigma(-\alpha) < \frac{1}{2} \iff |\delta| < \frac{1 + e^{\alpha}}{2}. \quad (6)$$

Since 6502 offsets satisfy $|\delta| \le 128$, $\alpha \ge 6$ suffices (bound greater than 202).

For a distance-softmax byte read with selected-address

weight w_0*,*

$$|\mathrm{read}_T - r_a| \le 255(1 - w_0) \le \frac{510e^{-1/T}}{1 - e^{-1/T}} < \frac{1}{2} \quad (T \le 0.14). \tag{7}$$

The bound is approximately 0.40 *at* $T = 0.14$*. Thus* $\alpha \ge 6$*,* $T \le 0.14$ *suffice independently of the executed program.*

Proof sketch. A branch differs from its hard target by $|\delta|\sigma(-\alpha)$; incorrect read-address weights are bounded by a two-sided geometric series. Both deviations below $1/2$ round to the hard integers, so induction gives whole-trajectory equality. The full proof is in the supplement. □

At $\alpha = 6$, $T = 0.14$, FULL was bit-exact over 6,000 instructions on Pong, Breakout, and Space Invaders (supplement). LEAKY uses $\alpha = 5.5$, $T = 0.145$ just outside the sufficient region, with mild forward divergence.

Corollary 1. *STE* (4) *makes full-system forward error zero for all finite* $\alpha, T > 0$*, irrespective of the bounds above. Its backward derivative is that of the chosen soft computation, not the hard machine's derivative.*

5 Mechanism, Intervention, Explanation

The known mechanism is the bit-exact hard transition map Φ_{H}. Ground truth for explanation is an *exact intervention*, not a gradient. For a ROM byte, RAM cell, register, or input u, replace its value by v and rerun the hard machine with all other conditions fixed:

$$\Delta y(u; v) = \Phi_{\mathrm{H}}(\ldots, \mathrm{do}(u := v), \ldots) - y. \tag{8}$$

Here Φ_{H} denotes execution to the observation time and y the original output.

We distinguish four output regimes. *Content* comprises register, colour, and graphics-bit values, where selected-value gradients are valid. *Position* comprises sprite coordinates set by strobe timing, where naive gradients through indices are provably zero. *Event* comprises collision latches and lost lives, also with zero naive gradients. *Score* records game scores.

For a method M, faithfulness $F \in [0, 1]$ measures correlation-based agreement between its per-cause attribution $a_M(u)$ and the exact intervention magnitude $|\Delta y(u; v)|$: $F = 1$ denotes exact agreement, $F = 0$ chance. Naive gradients, STE surrogates, samplers, and fully relaxed gradients are *methods under test*, never the reference. Exact forward equivalence does not establish their faithfulness; comparison with interventions does.

6 Evaluation Methodology

We evaluate 64 ALE games in the XITARI configuration. Canonical DQN used 49 games (Mnih et al. 2015); later work used Atari-57 (Machado et al. 2018). Each game follows the standard ALE boot (60 NOOP frames, four resets), then a fixed action stream. We compare all 128 RAM bytes on each of 30 frames and the full 210×160 frame buffer on each of 60 frames. RAM- and pixel-exactness require zero differences on every comparison. Claims concern these tested trajectories, not all possible executions.

For conformance, we patched XITARI's Superchip RAM seed from `time(NULL)` to zero, matching both ports. The patch ships with the code. One step denotes one CPU instruction, not an ALE frame. CPU measurements use an Apple M1 Max core; GPU measurements use a GTX 1080 Ti / Quadro RTX 5000 with peak batch $N \approx 4096$. Batched JAXTARI uses the soft CPU core with no TIA, a single ROM window, and only the RIOT timer. Its hard path is eager and cannot be GPU-batched. Full-system steps run eagerly on CPU in both ports; Zygote reverse-mode throughput is measured in JUTARI. The JAX GPU numbers remain soft-CPU-core only.

7 Results

Both ports reproduce XITARI on all 64 games: 64/64 RAM-exact over 30 frames and 64/64 pixel-exact over 60 frames.

In JUTARI, full-system SOFT-STE equals HARD on every instruction across 64 ROMs (3,000 each, starting 20 frames into play), with zero uncorrected soft-handler disagreements in CPU registers or RAM. In JAXTARI, the same holds on 9 ROMs covering every exercised mapper family (1,500 instructions each, after standard boot and 20 frames). The JUTARI checks exposed and fixed functional `BRK` and RAM-executed-code bugs. In JAXTARI, `jax.grad` through 100 full-system steps reaches ROM bytes.

The full-system soft step reaches $\sim$146,000 instructions/s on CPU. Zygote reverse-mode gradients are practical for 50–400-instruction windows, at a few hundred instructions/s. The batched JAXTARI soft CPU core reaches $\sim$3.1M instructions/s on GPU with $\sim 5\%$ gradient overhead; this is not full-machine differentiable rollout throughput.

Active Development Time

The version-control log records 373 commits over 29 days; splitting at three-hour gaps gives 52 sessions totalling 136.8 hours of active development wall-time. This is an order-of-magnitude estimate, not a controlled trial or measure of person-effort or compute. Coding agents wrote and debugged most of the $\sim$18,700 source lines under automated conformance tests (details in the supplement).

Proof of Concept: Gradients and Interventions

Figure 5 compares gradients with exact interventions. Starting 35 s after boot ($x = 35$), we hold RIGHT for 81 frames to reach the midpoint $x = 76$ of the measured range $[35, 117]$, avoiding wall clipping. The scene is pixel-exact against the reference. The joystick drives position directly through the sampler, bypassing ROM controller-reading code. Naive differentiation gives zero; the bilinear sampler localises screen gradients to cannon edges.

In panel (d), each series is normalised by its largest magnitude because units differ. The sampler gives RIGHT $+1.23$, LEFT -1.23, and UP/DOWN zero, identically for SOFT-STE and relaxed settings $(\alpha, T) = (6, 0.14), (5.5, 0.145)$ (maximum difference zero). Naive gradients vanish throughout. Hard-emulator interventions give RIGHT $+0.5$ and LEFT -0.5 px/frame, with UP/DOWN zero; only cannon

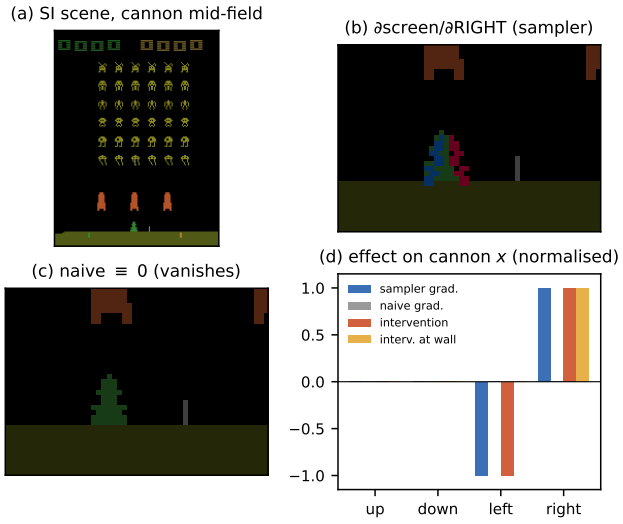


Figure 5: Space Invaders at the cannon midpoint ($x = 76$). (a) Pixel-exact scene. (b) Bilinear (triangular) sampler gradient $\partial\text{screen}/\partial\text{RIGHT}$, nonzero only at cannon edges. (c) Exactly zero naive gradient through integer sprite position. (d) Four normalised directional-effect series: sampler and naive gradients, and exact hard-emulator interventions (8 frames versus NOOP) at the midpoint and at the 35 s left-wall scene (LEFT has zero effect).

Table 3: Faithfulness against exact interventions (42 games; 95% bootstrap CI over games).

Method	Overall F	Position-only F
Occlusion (intervention)	0.69 [0.62, 0.75]	0.62
Vanilla saliency (gradient)	0.38 [0.29, 0.47]	0.12

pixels change (rows 186–195; 56 pixels). Thus the sign/zero pattern agrees away from the wall, but the surrogate magnitude and joystick scale are not game dynamics. At the wall, LEFT causes no pixel changes despite the same nonzero sampler gradient. Exact interventions, not gradients, provide the reference.

Scoring Attribution Methods

A companion study scores 30 methods across 42 ALE games: eight neuroscience-style analyses, 12 attribution methods, and ten mechanistic- interpretability methods. Faithfulness compares per-cause attribution with exact hard-machine intervention effects. Table 3 reports 95% bootstrap confidence intervals over games.

A program-randomisation sanity check (Adebayo et al. 2018) scrambles the ROM so the program boots to a different state. Yet content-path saliency remains unchanged (cosine $\sim$1) on all six core games, failing the check. A bilinear sampler restores non-zero position gradients, but faithfulness improves on only a few games, including Pong ($0 \to 0.79$) and Q*bert ($0 \to 0.53$). Non-zero gradients therefore do not establish faithful explanations.

8 Discussion

The ports reproduce all tested reference trajectories. SOFT-STE is forward-exact by construction (Theorem 1); the rounded relaxed variant is also exact within the conditions of Proposition 1. Neither result guarantees attribution faithfulness, as the intervention tests demonstrate.

The authors designed the hard/soft split and full-system STE step, the stored-switch (max-pooling) view restricting surrogates to discrete indices, the three conformance harnesses using two independent ports as the oracle, and the program-independent α/T bounds. Agents wrote and debugged most code under this oracle. Conformance exposed XITARI's time-dependent Superchip RAM seed; the deterministic seed-zero patch ships with the code.

Limitations and next steps. Conformance covers tested trajectories, not all executions. Audio is absent. JUTARI gradients are practical for short windows; GPU batching applies only to the soft CPU core, not full-system differentiation. The next step is to intervene on a game variable, re-render bit-exactly, and measure the resulting change in a learned agent's action. Calibration on the known machine must come first.

9 Conclusion

We provide a complex, fully known machine for mechanism-grounded XAI. Two Atari VCS ports reproduce the reference on 64 tested games. A full-system soft step preserves exact forward execution while supplying surrogate gradients. Exact interventions expose where those gradients agree with the known mechanism and where they fail. This provides a reference for testing explanations rather than relying on their plausibility.

Ethical Statement

This work involves no human subjects, personal data, or sensitive information. The commercial Atari ROMs are used only as fixed inputs; they are copyrighted and *not* redistributed—we release SHA-256 hashes for identification only. The agentic method is dual-use; pairing autonomy with a strict, automatically checked conformance oracle mitigates its central risk, silent divergence from the reference, without eliminating it.

References

Adebayo, J.; Gilmer, J.; Muelly, M.; Goodfellow, I.; Hardt, M.; and Kim, B. 2018. Sanity Checks for Saliency Maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 9505–9515.

Atrey, A.; Clary, K.; and Jensen, D. 2020. Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The Arcade Learning Environment: An Evaluation Platform for General Agents. *Journal of Artificial Intelligence Research*, 47: 253–279.

Bengio, Y.; Léonard, N.; and Courville, A. 2013. Estimating or Propagating Gradients through Stochastic Neurons for Conditional Computation. arXiv:1308.3432.

Bezanson, J.; Edelman, A.; Karpinski, S.; and Shah, V. B. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59(1): 65–98.

Bradbury, J.; Frostig, R.; Hawkins, P.; Johnson, M. J.; Leary, C.; Maclaurin, D.; Necula, G.; Paszke, A.; VanderPlas, J.; Wanderman-Milne, S.; and Zhang, Q. 2018. JAX: Composable Transformations of Python+NumPy Programs. http://github.com/google/jax. Software.

Chattopadhyay, A.; Sarkar, A.; Howlader, P.; and Balasubramanian, V. N. 2018. Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 839–847. IEEE.

Cheng, Z.; Yu, J.; and Xing, X. 2025. A Survey on Explainable Deep Reinforcement Learning. *arXiv preprint arXiv:2502.06869*.

Dalton, S.; Frosio, I.; and Garland, M. 2020. Accelerating Reinforcement Learning through GPU Atari Emulation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33. ArXiv:1907.08467; code at https://github.com/NVlabs/cule.

DeepMind. 2016. Xitari: An Arcade Learning Environment Fork. https://github.com/google-deepmind/xitari. Accessed: 2026-06-16.

Delfosse, Q.; Blüml, J.; Gregori, B.; Sztwiertnia, S.; and Kersting, K. 2023. OCAtari: Object-Centric Atari 2600 Reinforcement Learning Environments. arXiv:2306.08649.

Delfosse, Q.; Emunds, R.; Seitz, P.; Wette, S.; Blüml, J.; and Kersting, K. 2026. JAXAtari: A High-Performance Framework for Reasoning Agents. https://github.com/k4ntz/JAXAtari. Software; accessed 2026-06-19.

Freeman, C. D.; Frey, E.; Raichuk, A.; Girgin, S.; Mordatch, I.; and Bachem, O. 2021. Brax—A Differentiable Physics Engine for Large Scale Rigid Body Simulation. *arXiv preprint arXiv:2106.13281*.

Graves, A.; Wayne, G.; and Danihelka, I. 2014. Neural Turing Machines. arXiv:1410.5401.

Greydanus, S.; Koul, A.; Dodge, J.; and Fern, A. 2018. Visualizing and Understanding Atari Agents. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, 1792–1801. PMLR.

Gupta, R.; Arcuschin, I.; Kwa, T.; and Garriga-Alonso, A. 2024. InterpBench: Semi-Synthetic Transformers for Evaluating Mechanistic Interpretability Techniques. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Innes, M. 2019. Don't Unroll Adjoint: Differentiating SSA-Form Programs. arXiv:1810.07951.

Jaderberg, M.; Simonyan, K.; Zisserman, A.; and Kavukcuoglu, K. 2015. Spatial Transformer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28.

Jang, E.; Gu, S.; and Poole, B. 2017. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*.

Jonas, E.; and Kording, K. P. 2017. Could a Neuroscientist Understand a Microprocessor? *PLOS Computational Biology*, 13(1): e1005268.

Kontogiannis, A.; and Vouros, G. A. 2023. XDQN: Inherently Interpretable DQN through Mimicking. arXiv:2301.03043.

Lindner, D.; Kramár, J.; Farquhar, S.; Rahtz, M.; McGrath, T.; and Mikulik, V. 2023. Tracr: Compiled Transformers as a Laboratory for Interpretability. In *Advances in Neural Information Processing Systems*.

Machado, M. C.; Bellemare, M. G.; Talvitie, E.; Veness, J.; Hausknecht, M.; and Bowling, M. 2018. Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents. *Journal of Artificial Intelligence Research*, 61: 523–562.

Maier, A.; Syben, C.; Lasser, T.; and Riess, C. 2019a. A Gentle Introduction to Deep Learning in Medical Image Processing. *Zeitschrift für Medizinische Physik*, 29(2): 86–101.

Maier, A. K.; Syben, C.; Stimpel, B.; Würfl, T.; Hoffmann, M.; Schebesch, F.; Fu, W.; Mill, L.; Kling, L.; and Christiansen, S. 2019b. Learning with Known Operators Reduces Maximum Error Bounds. *Nature Machine Intelligence*, 1(8): 373–380.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Nikulin, D.; Ianina, A.; Aliev, V.; and Nikolenko, S. 2019. Free-Lunch Saliency via Attention in Atari Agents. arXiv:1908.02511.

Qing, Y.; Liu, S.; Song, J.; Zhou, Y.; Chen, K.; Wang, H.; and Song, M. 2022. A Survey on Explainable Reinforcement Learning: Concepts, Algorithms, and Challenges. *arXiv preprint arXiv:2211.06665*.

Raissi, M.; Perdikaris, P.; and Karniadakis, G. E. 2019. Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *Journal of Computational Physics*, 378: 686–707.

Saulières, L. 2025. A Survey of Explainable Reinforcement Learning: Targets, Methods and Needs. *arXiv preprint arXiv:2507.12599*.

Selvaraju, R. R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; and Batra, D. 2020. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *International Journal of Computer Vision*, 128(2): 336–359. Originally arXiv:1610.02391 (2016).

Stella Team. 2024. Stella: A Multi-Platform Atari 2600 VCS Emulator. https://stella-emu.github.io. Accessed: 2026-06-16.

Such, F. P.; Madhavan, V.; Liu, R.; Wang, R.; Castro, P. S.; Li, Y.; Zhi, J.; Schubert, L.; Bellemare, M. G.; Clune, J.; and Lehman, J. 2019. An Atari Model Zoo for Analyzing, Visualizing, and Comparing Deep Reinforcement Learning Agents. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 3260–3267.

Vouros, G. A. 2023. Explainable Deep Reinforcement Learning: State of the Art and Challenges. *ACM Computing Surveys*, 55(5): 1–39.

Würfl, T.; Ghesu, F. C.; Christlein, V.; and Maier, A. 2016. Deep Learning Computed Tomography. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 432–440. Springer.

Yang, M.; and Kim, B. 2019. BIM: Towards Quantitative Evaluation of Interpretability Methods with Ground Truth. *arXiv preprint arXiv:1907.09701*.